

Instagram Reel Metrics — Beginner-Friendly Guide

A small local web app that fetches **real Instagram Reel statistics** — likes, comments, views, shares, saves, reposts — and can list every reel of any public account.

This README is written for students. No prior knowledge of Instagram's APIs is assumed.

Table of Contents

1. What is this project?
 2. The big idea: Instagram has TWO doors
 3. What is `instagrapi` ?
 4. How the whole thing fits together
 5. The two files in this project
 6. The request lifecycle (step by step)
 7. Logging in and sessions
 8. Single Reel mode vs Profile Reels mode
 9. Running it on your machine
 10. Common errors and what they mean
 11. Glossary
-

1. What is this project?

Imagine you're a social-media analyst, and your boss asks:

"Tell me how many likes, comments, and views this Instagram reel got — and oh, also pull the same info for every reel of `@atiazuhair`."

You could open each reel and read the numbers by hand. **Slow.** You could ask Instagram nicely. **They won't answer.** So instead, we built a tiny tool that:

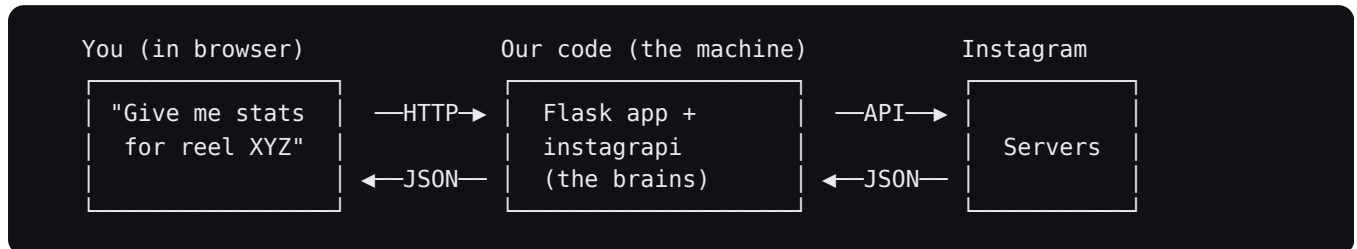
- Pretends to be the Instagram mobile app.
- Logs in once with your username and password.
- Pulls the numbers automatically.

- Shows them in a nice browser interface.

That tool is this project. It runs **locally on your computer** — nothing is uploaded to a server somewhere.

The big-picture analogy

Think of the project like a **vending machine for Instagram numbers**:

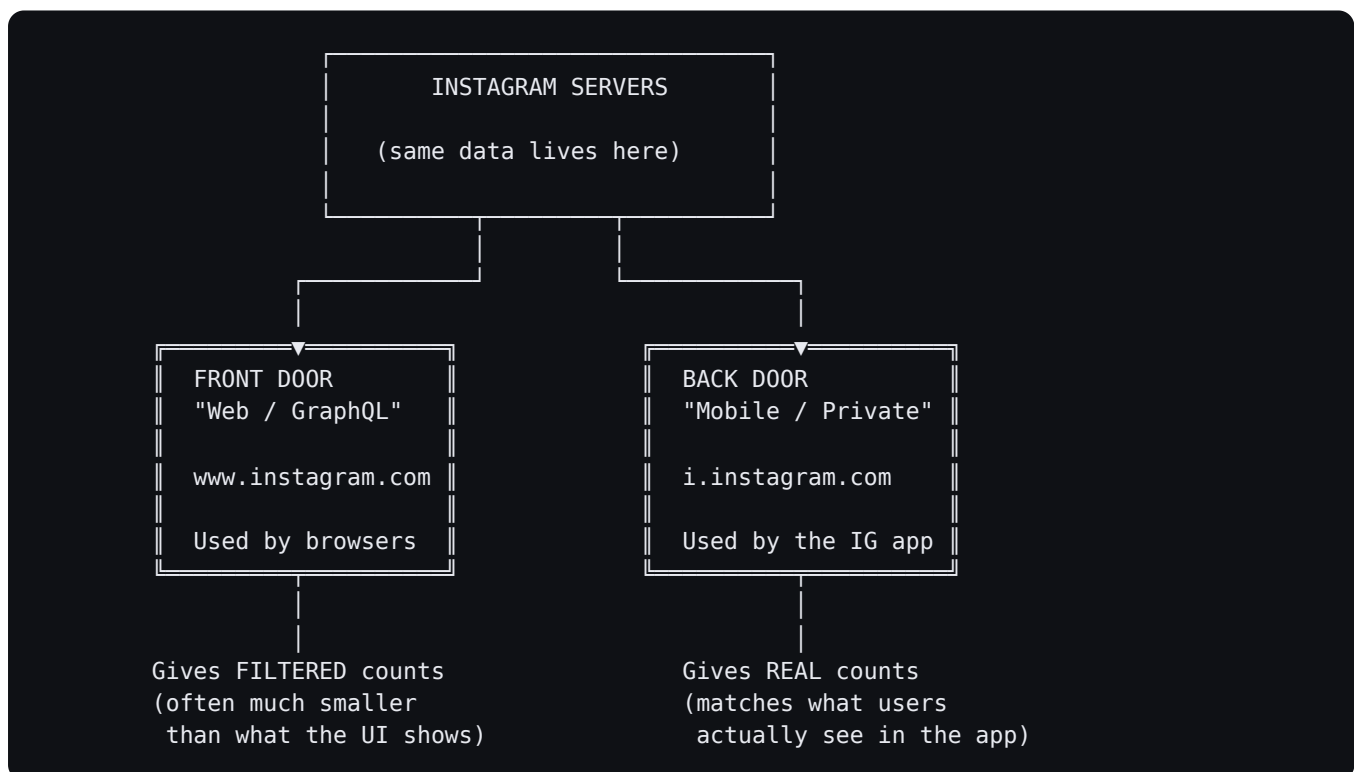


You drop a request in the slot, the machine fetches and decodes the answer, you get a nice display.

2. The big idea: Instagram has TWO doors

This is the most important concept in the whole project. Pay attention.

Instagram does NOT have one single way to ask for data. It has **two completely separate APIs**, like two doors to the same building:



Cashier analogy

Imagine two cashiers in the same supermarket counting how many customers visited today:

- **Front-door cashier (Web API):** "I can only count customers I personally saw walk in. So my count is 1,367."
- **Back-door cashier (Mobile API):** "I have access to the security camera footage of every entrance. The real number is 5,115."

Both cashiers work for the same supermarket. They just have different views of the same building. When we want the *real* number, we have to ask the back-door cashier.

Key insight: Older libraries like `instaloader` only know the front door. They return 1,367 likes when the real number is 5,115. That's why our app started giving wrong numbers — until we switched to the back-door approach.

3. What is `instagrapi` ?

`instagrapi` is a **Python library** (a collection of pre-written code) that knows how to talk to Instagram's back door.

How to explain it to a 10-year-old

Instagram's back door (the "mobile API") has a secret knock. If you knock the right rhythm, the door opens and you get the real numbers. If you knock wrong, the door stays shut and they tell you "go away".

`instagrapi` is the **rhythm book**. It memorises the right knock — every header, every random ID, every cookie — and replays it so Instagram thinks we're a real iPhone running the Instagram app.

What `instagrapi` actually does for us

What you write	What <code>instagrapi</code> does behind the scenes
<code>cl.login("user", "pass")</code>	Sends ~10 mobile-style HTTP requests with the correct device IDs, randomised UUIDs, encrypted password, etc. Stores cookies.
<code>cl.media_pk_from_code("DXXIFFy...")</code>	Turns the short URL slug into Instagram's internal number ID for that post.
<code>cl.user_clips_v1(user_id, amount=20)</code>	Pages through Instagram's "give me this user's reels" endpoint and returns 20 reels worth of data.
<code>cl.private_request("media/<pk>/info/")</code>	Makes a raw mobile-API call to a custom endpoint. We use this to get the FULL set of metrics (likes, comments, plays, shares, saves, reposts).

Why we don't write all that ourselves

If we tried to talk to `i.instagram.com` from scratch, we'd need to handle:

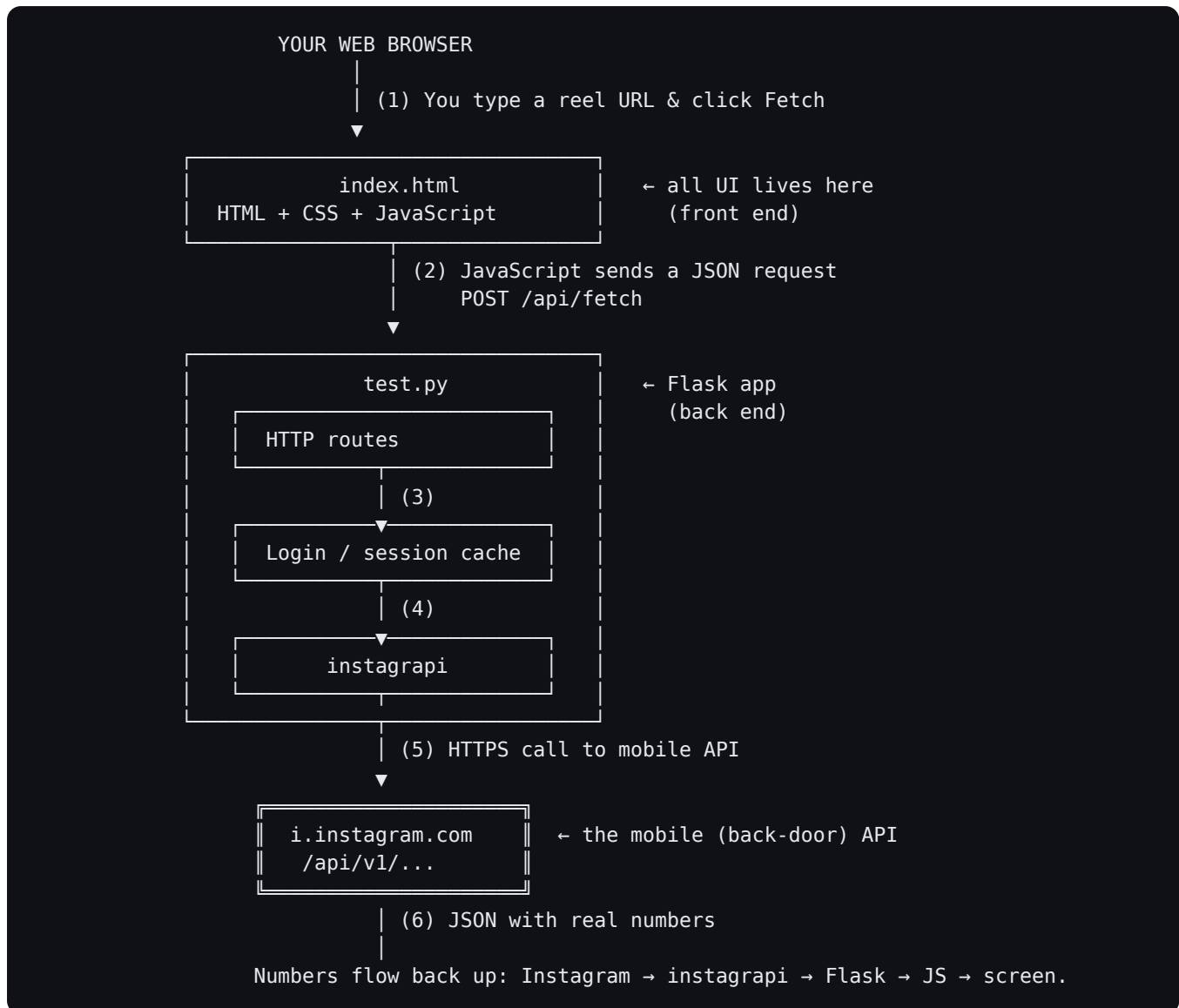
- Generating a random Android/iPhone device fingerprint that Instagram accepts.
- Encrypting the password with a public key Instagram rotates.
- Adding ~20 special headers (`X-IG-App-ID` , `X-IG-Capabilities` , ...).
- Handling cookies, login challenges, rate-limit retries.

That's months of work. `instagrapi` does it all for us in two lines:

```
cl = Client()
cl.login("username", "password")
```

4. How the whole thing fits together

Here is the project at a glance:



So whenever you click a button in the browser, it really just bounces a JSON message down this stack and a JSON answer back up.

5. The two files in this project

That's it. Just two files.

```
Practive Frontend/
├── test.py          (the "brain" – Python / Flask backend)
└── index.html      (the "face" – HTML + CSS + JavaScript front end)
```

Why split them?

File	Job	Analogy
<code>index.html</code>	Show buttons, take input, draw results.	The steering wheel and dashboard of a car.
<code>test.py</code>	Talk to Instagram, do all the heavy lifting.	The engine of the car.

You can change the dashboard (colors, layout, text) without touching the engine — and vice versa. That's good software design.

Inside `test.py` (the brain)

`test.py` is divided into **7 sections** clearly marked with banner comments:

```
1. Imports & module setup      — grab the tools we need
2. Session management         — log in once, remember it
3. URL / shortcode parsers    — accept any input format
4. Scraping primitives        — ask Instagram for data
5. Error mapping              — turn errors into friendly messages
6. HTTP routes                — what the browser calls
7. Entry point                — start the server
```

Each section is a chapter. You can jump to chapter 6, for example, and understand what the API endpoints do without reading the rest first.

6. The request lifecycle (step by step)

Let's trace what happens when you click "**Fetch Metrics**" on a single reel.

STEP 1 – Browser sends JSON

```
POST /api/fetch
{ "username": "you", "password": "...", "shortcode": "DWap..." }
```

STEP 2 – Flask receives + validates

- Body has all required fields? ✓
- If not → return 400 Bad Request

STEP 3 – get_client()

└ Already logged in this run?	—— yes →	reuse client
└ Have a saved session on disk?	—— yes →	load + verify
└ Otherwise	—— yes →	password login

STEP 4 – Parse the input

```
extract_shortcode("https://.../reel/DWap.../?igsh=...")
→ "DWap..."

cl.media_pk_from_code("DWap...")
→ 3862581467293681892 (internal numeric ID)
```

STEP 5 – fetch_single_media()

```
cl.private_request("media/3862.../info/")
→ talks to i.instagram.com
→ returns a giant JSON object with ALL stats
```

We pick the fields we want:

```
like_count, comment_count, play_count,
reshare_count, save_count, media_repost_count
```

STEP 6 – Send JSON back to the browser

```
{ "ok": true,
  "likes": 5115, "comments": 122, "views": 303585,
  "shares": 573, "saves": 511, "reposts": 34, ... }
```

```
STEP 7 – Browser draws the numbers
```

```
JavaScript fills each metric card on the page.  
You see the answer.
```

That's the full journey — about 1–2 seconds from click to display.

7. Logging in and sessions

Every time you ask Instagram for something, you need to prove who you are. That's annoying if you have to type your password every request, so we use **sessions**.

What's a session?

A session is like a **stamp on your wrist at a theme park**:

- You pay once at the entrance and get the stamp.
- For the rest of the day, you can leave and come back without paying again.
- At the end of the day, the stamp wears off and you'd have to pay again next time.

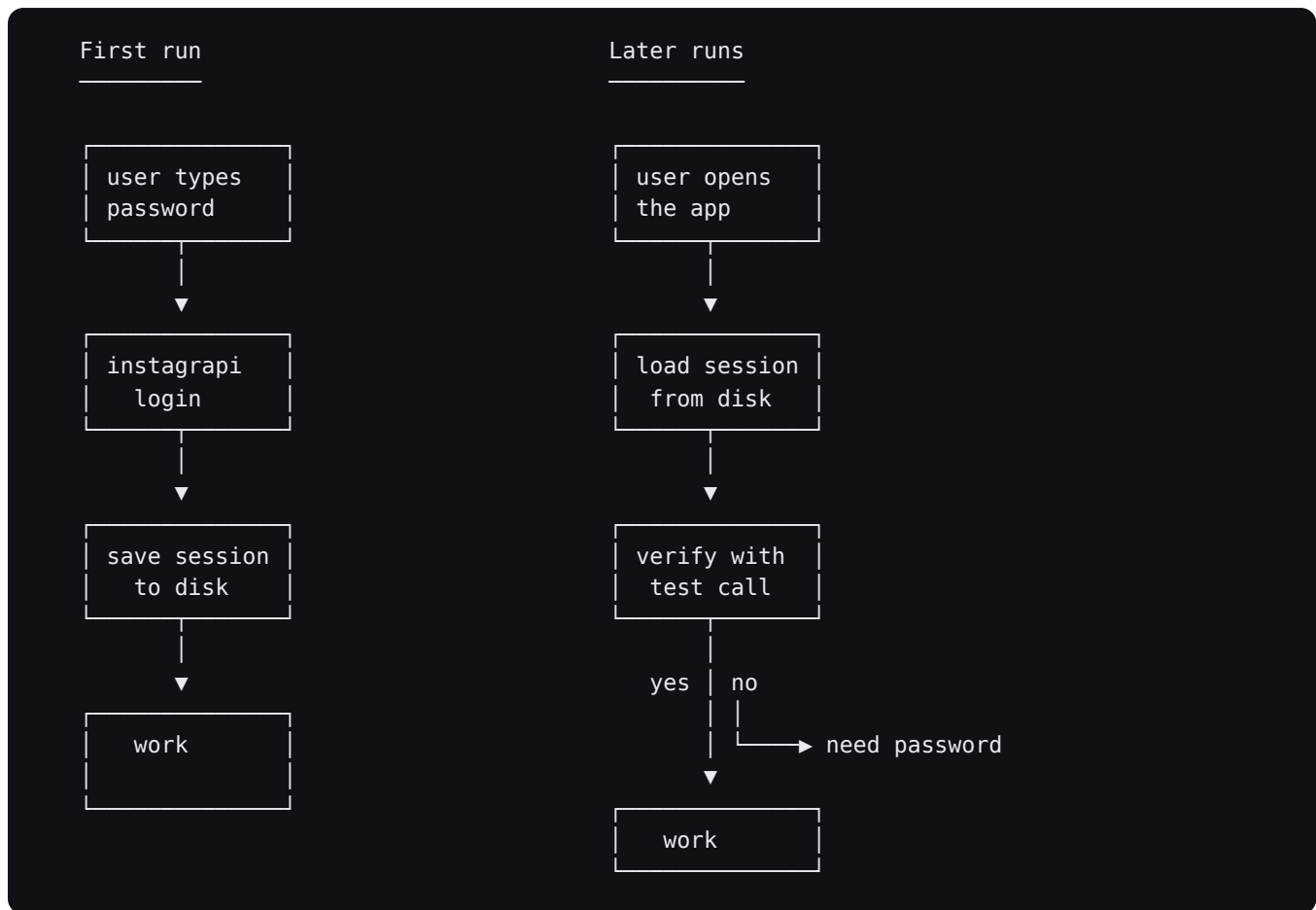
When `instagrapi` logs in successfully, Instagram gives our client a "stamp" (a bunch of cookies + device IDs). We save those to a file:

```
.ig_sessions/instagrapi-<your_username>.json
```

Next time you open the app:

1. We load that file.
2. We make one cheap test call (`get_timeline_feed`) to see if the stamp is still valid.
3. If yes → no password needed.
4. If no (expired/revoked) → you have to type your password again.

Session lifecycle diagram



8. Single Reel mode vs Profile Reels mode

The app has two tabs. They use different scraping strategies because they have different needs.

Single Reel

- **One reel** → make **one** rich request → get **everything** (shares, saves, reposts too).
- Slow per-reel but you only do it once.

Profile Reels

- **Many reels** → can't afford one rich request per reel (Instagram would rate-limit us).
- Instead: make **one bulk** "give me this user's reels" call.
- That bulk call returns a lighter dataset (no shares/saves/reposts) but gives us likes/comments/views for every reel in one go.

Single Reel

1 request → 1 reel

full detail
(likes + comments + views
+ shares + saves + reposts)

Profile Reels

1 request → N reels

light detail
(likes + comments + views)

Why the difference? Instagram limits how many requests you can make per minute. If we do **N** rich requests for a profile with 200 reels, we'd burn our limit instantly. Doing 1 bulk request keeps us under the radar.

9. Running it on your machine

The first time:

```
# 1. Make a virtual environment so we don't pollute system Python
python3 -m venv .venv
source .venv/bin/activate          # Linux / macOS
# .venv\Scripts\activate           # Windows

# 2. Install the two libraries we need
pip install flask instagrapi

# 3. Start the server
python test.py
```

Then open your browser to <http://127.0.0.1:5000>.

To run it again later, you only need:

```
source .venv/bin/activate
python test.py
```

10. Common errors and what they mean

You see this	What it actually means	What to do
<code>ChallengeRequired</code>	Instagram doesn't recognise the device; it wants email/SMS verification.	Open the real Instagram app, confirm "It's me", retry.
<code>BadPassword</code>	Wrong password (or Instagram thinks it's a bot).	Type it carefully. If correct, wait an hour and try again.
<code>LoginRequired</code>	Your saved session expired.	Provide your password again — we'll save a fresh session.
<code>PleaseWaitFewMinutes</code>	You sent too many requests too fast.	Wait 5–10 minutes. Use smaller <code>Max Reels</code> values.
<code>UserNotFound</code>	The username/shortcode doesn't exist.	Check the spelling.
<code>PrivateAccount</code>	The profile is private and your account doesn't follow it.	Follow the account first.

11. Glossary

API — A way for one program to talk to another. Like a menu at a restaurant: it lists what you can order, and how to order it.

Cookie — A tiny piece of data the server tells your client to remember. Instagram uses cookies to know "you logged in 5 minutes ago, no need to log in again."

Endpoint — A specific URL on a server that does one thing. e.g.

`i.instagram.com/api/v1/media/<pk>/info/` is the "get info about a single post" endpoint.

Flask — A Python framework for building web apps. Tiny and easy. We use it to serve `index.html` and handle our 3 API routes.

GraphQL — One particular flavour of API used by Instagram's web (browser) version. Returns less data than the mobile API.

`instagrapi` — The Python library that talks to Instagram's mobile API on our behalf. See [Section 3](#).

Mobile API — Instagram's "back door" API used by the iPhone and Android apps. Gives us the real numbers.

`media_pk` — Instagram's internal numeric ID for a post (e.g. `3862581467293681892`). Different from the shortcode (`DWapWEyDAjk`) — they refer to the same post but are different formats.

Rate limit — A cap on how many requests you can make per minute. Goes up when you behave like a human, down when you behave like a bot.

Session — Your "stamp on the wrist" — a saved login state so you don't re-enter your password every time.

Shortcode — The slug at the end of a reel URL (`DWapWEyDAjk` in `instagram.com/reel/DWapWEyDAjk/`). Used as a human-friendly post identifier.

Virtual environment (`venv`) — An isolated copy of Python with its own libraries. Keeps your project's dependencies separate from the rest of your computer.

Final word

That's the whole project. Two files. Two APIs. One library doing the heavy lifting. If you understand:

1. *Instagram has two APIs and we want the back-door one,*
2. `instagrapi` *knocks on that back door for us,*
3. *Flask glues the browser to* `instagrapi`*,*

...then you understand the code. Everything else is just plumbing.

Happy scraping.